

ENTANDO 7.1 Tutorials ▾

Get Started ▶

Solution Templates ▶

Create Components ▶

Curate and Share Applications ▶

Compose an Application ▶

Consume an Application ▶

Configure and Customize an Application ▾

Default Database

External Database

Entando Docker Image

Backing Up and Restoring Keycloak

Change Default Datasource

Manage NGINX

Verify the NGINX Ingress Install

Customize the NGINX Configuration

Add the cert-manager for TLS Support

Installation

Configuration

cert-manager References

Plugin Configuration

Plugin Environment Variables

Configure the Entando App Context

Manage NGINX

There are environments where it's useful to use NGINX rather than the default ingress controller. This page shows how to verify and refine your NGINX configuration.

See the following install guides if needed:

- Amazon Elastic Kubernetes Service (EKS)
- Azure Kubernetes Service (AKS)
- Google Kubernetes Engine (GKE)

Verify the NGINX Ingress Install

To verify that the ingress is working properly you can set up a test application.

- Create a simple application:

```
kubectl create deployment hello-server --image=us-docker.pkg.dev/google-samples/containers
```

- Expose the `hello-app` deployment as a Service:

```
kubectl expose deployment hello-server --type LoadBalancer --port 80 --target-port 8080
```

- Create this `ingress-resource.yaml` file:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-resource
  annotations:
    kubernetes.io/ingress.class: nginx
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
spec:
  rules:
    - http:
        paths:
          - path: /hello
            pathType: Prefix
        backend:
          service:
            name: hello-server
            port:
              number: 80
```

- Create the Ingress Resource:

```
kubectl apply -f ingress-resource.yaml
```

- Verify that the Ingress Resource has been created:

```
kubectl get ingress ingress-resource
```

It may take several minutes to populate the `Address` .

- Verify access to the web application using the `EXTERNAL-IP/hello` address of the `nginx-ingress-controller` . You should see the following:

```
Hello, world!
Version: 1.0.0
Hostname: hello-app
```

Note that you will need the EXTERNAL-IP address of your ingress controller to configure the application.

- Verify that you configured the ingress class in the Operator `ConfigMap` so Entando knows which ingress controller should be used:

```
entando.ingress.class: "nginx"
```

- To reduce costs, remove the test deployment, service, and ingress:

```
kubectl delete deploy/hello-server service/hello-server ing/ingress-resource
```

Customize the NGINX Configuration

There are situations where the default NGINX ingress configuration isn't optimized for Entando, e.g. JWT tokens can be too large or `proxy-buffer-size` can be too small. A `502 Bad Gateway` error may indicate that the config needs to be modified.

The NGINX controller can be configured for the entire cluster by editing the default NGINX `ConfigMap` , called `ingress-nginx-controller` in the `ingress-nginx` namespace. Add the following to the data parameter:

```
apiVersion: v1
data:
  allow-snippet-annotations: "true"
  proxy-buffer-size: 24k
kind: ConfigMap
```

Production environments may require additional common annotations:

```
nginx.ingress.kubernetes.io/client-max-body-size: 200m # the maximum allowed size of the nginx.ingress.kubernetes.io/proxy-body-size: 200m # to upload large files (default is 1k)
nginx.ingress.kubernetes.io/proxy-buffer-size: 64k # for the Keycloak auth-token (default is 1k)
nginx.ingress.kubernetes.io/proxy-read-timeout: "600" # to increase the timeout (in seconds)
```

Sticky sessions may be useful for `entando-de-app` deployments with multiple replicas. If you [set up clustering](#), the following options enable sticky sessions in NGINX:

```
nginx.ingress.kubernetes.io/affinity: cookie
nginx.ingress.kubernetes.io/affinity-mode: persistent
nginx.ingress.kubernetes.io/session-cookie-name: ROUTE
nginx.ingress.kubernetes.io/session-cookie-secure: "true"
```

Add the `cert-manager` for TLS Support

Follow the instructions below to install and configure `cert-manager` in Kubernetes environments.

Installation

Create a namespace dedicated to `cert-manager` :

```
kubectl create ns cert-manager
```

Complete the installation:

```
kubectl apply -f https://github.com/jetstack/cert-manager/releases/download/v1.7.0/cert-manager.yaml
```

Configuration

To enable `cert-manager` to generate certificates, add these annotations to the ingress:

- `cert-manager.io/issuer: "[name of the issuer]"` for namespace-based issuers
- `cert-manager.io/cluster-issuer: "[name of cluster issuer]"` for cluster-wide issuers

and modify `spec` :

```
spec:
  tls:
    - hosts:
        - example.example.com
      secretName: quickstart-example-tls # this Secret will be autogenerated by cert-manager
```

Namespace Level Issuer

Use the following configuration when deploying an issuer per namespace. This is useful for higher levels of customization.

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    # The ACME server URL
    server: https://acme-v02.api.letsencrypt.org/directory
    preferredChain: "ISRG Root X1"
    # Email address used for ACME registration
    email: <your email>
    # Name of a secret used to store the ACME account private key
    privateKeySecretRef:
      name: letsencrypt-prod
    # Enable the http-01 challenge provider
    solvers:
      - http01:
          ingress:
            class: nginx
```

Cluster Level Issuer

Use the following configuration when deploying an issuer per cluster:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod-cluster
  namespace: cert-manager
spec:
  acme:
    # The ACME server URL
    server: https://acme-v02.api.letsencrypt.org/directory
    preferredChain: "ISRG Root X1"
    # Email address used for ACME registration
    email: <your email>
    # Name of a secret used to store the ACME account private key
    privateKeySecretRef:
      name: letsencrypt-cluster-prod
    # Enable the http-01 challenge provider
    solvers:
      - http01:
          ingress:
            class: nginx
```

`cert-manager` References

- [Installation](#)
- [Supported releases](#)